

seqbench: Anytime-Valid Sequential Benchmarking of Algorithms in R

by André Leite, Raydonal Ospina, and Cristiano Ferraz

Abstract Comparing two algorithms on a benchmark is a sequential experiment: results are inspected as instances are run, budgets are adapted and the comparison stops when it looks settled, and fixed-sample tests lose their guarantees under this use. seqbench treats the comparison as a sequential experiment on paired losses and maintains a confidence sequence for the mean paired difference that remains valid under optional stopping, together with a pre-registered margin of practical equivalence. It issues one of three declarations, continues, or ends inconclusively when the budget is exhausted; all declarations share one error level. We describe the protocol, the package’s result contract and refusals, and a simulation study with 2,000 replications per cell against fixed-sample tests, an invalid repeatedly inspected test and the anytime-valid Gaussian confidence sequence of safestats. The distribution-free boundaries never exceeded 0.45% false declarations; the repeatedly inspected t-test reached 38%; the Gaussian sequence with a known variance needed 1.4 to 20 times fewer instances, a cost the package makes visible. Three applications show the workflow.

1 Introduction

Algorithm comparisons in optimisation, machine learning and computational statistics are rarely fixed-sample experiments. Instances are run in batches; intermediate results are inspected; more seeds are added where the outcome looks close; the study stops when one method “clearly” dominates or when the budget runs out. Every one of these habits invalidates a fixed-sample test: inspecting a *t*-test after each instance and stopping at the first rejection inflates the type-I error without bound (Armitage et al. (1969)), and the simulation study in Section 5 measures it at 38% for a nominal 5% in one of our scenarios. The problem is old and well understood in clinical trials; the benchmarking literature mostly ignores it. Best-practice guides recommend statistical tests (Bartz-Beielstein et al. (2020); Eimer et al. (2025)) but not sequential ones; racing methods used in algorithm configuration (Birattari et al. (2002); López-Ibáñez et al. (2016)) inspect repeatedly by design and explicitly disclaim error control.

The statistical tool that fixes this is the confidence sequence (Howard et al. (2021); Ramdas et al. (2023)): a sequence of intervals C_t with $P(\forall t : \mu \in C_t) \geq 1 - \alpha$, valid at every data-dependent stopping time. Waudby-Smith and Ramdas (2024) give tight confidence sequences for the mean of bounded random variables by betting; Choe and Ramdas (2024) apply confidence sequences to the paired comparison of two forecasters; the R packages *safestats* (Ly et al., 2026, 2024) and *seqcomp* (Alasgarli, 2026) implement sequential *t*-tests and forecast comparisons respectively.

seqbench packages this machinery into a *protocol for algorithm benchmarking*. Its contributions are practical, not methodological: an explicit population estimand for the comparison, pairing by instance with seeds treated as a cluster, a pre-registered margin of practical equivalence turned into a three-way decision rule that inherits the error control of the confidence sequence, cost accounting with an inconclusive outcome that is never converted into a decision, and a result contract that reports what was assumed and what was consumed. We are explicit about what this does *not* buy: the error control is a margin rule applied to an existing confidence sequence, and, as Section 5 shows, the distribution-free boundaries pay a substantial price in instances relative to a Gaussian sequence that knows the variance. The package makes that price visible instead of hiding it.

Section 2 states the problem and the assumptions; Section 3 the protocol and its guarantee; Section 4 the package; Section 5 the simulation study; Section 6 three applications; Section 7 discusses limitations and the road ahead.

2 The comparison as a sequential experiment

Estimand

Let P be a population of problem instances and S a distribution of seeds (random restarts, initialisations, resampling). Running algorithm $k \in \{A, B\}$ on instance i with seed s yields a loss $L_k(i, s)$, smaller being better; the loss is the user’s choice and the package does not interpret it. The paired difference is $D(i, s) = L_A(i, s) - L_B(i, s)$ and the estimand is the population mean

$$\mu = E_{i \sim P}[E_{s \sim S}[D(i, s)]].$$

A negative μ favours A. This is what “A is better than B on problems like these” means. It is *not* the mean over a fixed benchmark set (a finite-population target, for which sequential inference would sample without replacement), nor a probability of winning; both are legitimate targets and neither is the one addressed here.

Practical relevance

A margin $\delta > 0$, in loss units, is fixed before any data are seen and partitions the parameter space into three hypotheses that cover the line (Table 1).

Table 1: The three hypotheses defined by the margin.

Hypothesis	Condition	Reading
H_A	$\mu < -\delta$	A is relevantly better
H_0	$ \mu \leq \delta$	practically equivalent
H_B	$\mu > \delta$	B is relevantly better

The margin is a domain decision (one percentage point of accuracy, 0.1 points of optimality gap, one modularity point), recorded with the design and never tuned after inspection. This is the reading of a confidence interval used by equivalence testing (Schuirmann, 1987; Lakens, 2017) and by Bayesian regions of practical equivalence (Benavoli et al., 2017), made anytime-valid.

Assumptions

- **(A1) Independent instances.** Instances are drawn i.i.d. from P .
- **(A2) Seeds fixed by design.** The number of seeds for an instance and their values are chosen without looking at any loss of that instance; seeds are drawn from S .
- **(A3) Bounded losses.** Both losses lie in a known interval $[\ell_{lo}, \ell_{hi}]$, so $D \in [a, b] = [\ell_{lo} - \ell_{hi}, \ell_{hi} - \ell_{lo}]$.
- **(A4) No adaptive reuse.** A draw whose losses were observed does not receive new seeds and does not reappear.

Under (A1)–(A3) the per-instance seed-averaged difference, rescaled to $X_t = (\bar{D}_{i_t} - a) / (b - a) \in [0, 1]$, satisfies $E[X_t \mid \mathcal{F}_{t-1}] = \nu$ with $\nu = 1/2 + \mu / (b - a)$, which is what the confidence sequences below require; the X_t need not be identically distributed (the seed count may vary), and arbitrary dependence between the two algorithms and across seeds of the same instance is allowed. (A4) is what makes (A2) enforceable: adding seeds to an instance after seeing its losses makes X_t depend on past outcomes. The sequential unit is therefore the instance, not the evaluation.

Timeouts and failed runs are handled before the data reach the package: a penalised outcome (PAR- k : the cutoff time times k for unfinished runs, Hutter et al., 2009) keeps the loss bounded and changes the estimand to the mean penalised loss; censoring would keep the estimand and break boundedness. The package takes the first route and records the choice as the user’s.

3 The protocol and its guarantee

Confidence sequences for bounded means

On the $[0, 1]$ scale the package offers three confidence sequences from Waudby-Smith and Ramdas (2024), all with the time-uniform guarantee under the conditional-mean condition above: the predictable-plug-in Hoeffding sequence (a deterministic width, distribution-free and very conservative), the predictable-plug-in empirical-Bernstein sequence (variance adaptive) and the hedged betting sequence (the default, the tightest of the three). A fourth option, “naive_fixed”, is the fixed-sample t interval recomputed at every instance; it is *not* a confidence sequence, is flagged invalid in every output, and exists only as the negative control of Section 5. Since every set of one confidence sequence contains ν on the same event, the running intersection $C_t^\cap = \bigcap_{s \leq t} C_s$ is again a $1 - \alpha$ confidence sequence, and the decision rules use it.

Decision rule

With $C_t^\cap = [L_t, U_t]$ mapped back to loss units, checked in this order:

1. inconclusive with reason `cs_empty` if $U_t < L_t$ (possible with probability at most α ; never a decision);
2. A is relevantly better if $U_t < -\delta$;
3. B is relevantly better if $L_t > \delta$;
4. practically equivalent if $-\delta \leq L_t$ and $U_t \leq \delta$;
5. otherwise continue, unless the budget or the maximum number of instances is exhausted, in which case the outcome is inconclusive with reason `budget` or `n_max`.

Proposition (no false declaration, single α). Let C_t be a $1 - \alpha$ confidence sequence for ν . For every stopping time τ , including budget-driven ones, the probability that the procedure ever issues a false declaration, of any of the three kinds, is at most α .

Proof. On the event $E = \{\forall t : \nu \in C_t\}$, which has probability at least $1 - \alpha$, $L_t \leq \nu \leq U_t$ for all t ; each declaration condition then implies the corresponding hypothesis, so every declaration on E is correct, at every t and in particular at τ . The three false-declaration events are disjoint and contained in E^c . $\square$

The proof is one paragraph because the work is done by the confidence sequence. What the protocol adds is the reading: the three declarations are events on one simultaneous confidence set, so no split of α is needed (TOST needs none either; it is an intersection–union test), and the guarantee holds at the data-dependent time at which a benchmarker actually stops. “Inconclusive” carries no error; it is the complement of a declaration, and its probability is a design quantity.

Planning

For the Hoeffding sequence the half-width w_t is deterministic, and on the coverage event the correct declaration has been issued by $t^*(g) = \min\{t : 2w_t < g\}$, where g is the distance from ν to the nearest threshold on the $[0, 1]$ scale. This is computable before any data are collected from $(\alpha, \delta, \text{bounds})$ and a target distance, and it is what `planning_horizon()` returns; a variance-based approximation using the empirical-Bernstein width with a declared standard deviation is available as well. Both are conservative: typical stopping times are one half (Bernstein) to one quarter (betting) of the returned value in the regimes of Section 5, whose last part explains why.

4 The package

Workflow

```
library(seqbench)
set.seed(1)
design <- comparison_design(alpha = 0.05, margin = 0.02, bounds = c(0, 1),
                           boundary = "betting", budget = 800)
state <- initialize_comparison(design)

losses <- data.frame(instance = 1:50, seed = 1L,
                     loss_a = runif(50, 0.4, 0.8), loss_b = runif(50, 0.3, 0.7))
state <- update_comparison(state, losses) # one row per (instance, seed)
stopping_decision(state)

#> [1] "continue"
#> attr("reason")
#> [1] NA

glance(state)

#> # A tibble: 1 x 12
#>   boundary valid alpha margin n_instances n_evaluations cost estimate lower
#>   <chr>    <lg1> <dbl> <dbl>      <int>          <int> <dbl>   <dbl> <dbl>
#> 1 betting TRUE  0.05  0.02        50            50    50    0.112 -0.0404
#> # i 3 more variables: upper <dbl>, decision <chr>, stopping_reason <chr>
```

`update_comparison()` is incremental: rows of new instances can be fed at any time, in any batch size; seeds of the same instance are averaged into one observation; the procedure stops at the first instance that triggers a declaration and refuses data after that (the pre-registered protocol ends there),

with a warning listing what was not consumed. `comparison_report()` (also `summary()`) returns the full result contract, `tidy()` the trajectory (one row per instance with the interval, its running intersection, cost and the decision at that time), `glance()` a one-row summary, and `autoplot()` the confidence sequence against the number of instances with the equivalence band.

The result contract

Every comparison reports: the estimand (as a sentence), the estimate, the current confidence sequence, the boundary and its validity flag, the four assumptions in words, the decision and stopping reason, diagnostics (instances, evaluations, losses at the bounds, empty-set flag), resources consumed against the budget, the instance and seed identifiers in the order observed, and the package and R versions. The intent is that a reader can audit a decision without rerunning it.

Refusals

The package never converts invalid input into a valid-looking result. A loss outside the declared bounds is an error, not a clipped value; a missing loss is an error, not a dropped row; a negative or infinite cost is an error; a duplicated (instance, seed) pair is an error; an instance that reappears after its losses were observed is an error with a message explaining (A4); `paired = FALSE` is an error saying that unpaired designs are not implemented. Updating after a terminal decision is an error too.

Implementation notes

Each boundary is a small state machine with sufficient statistics, exposed through three kernels that sister packages can reuse: `boundary_init()` creates the state, `boundary_update()` absorbs one observation and `boundary_interval()` reads the current confidence set. The betting boundary keeps the log-capital on a grid of candidate means; its confidence set is the set of grid points whose capital is below $1/\alpha$, which Waudby-Smith and Ramdas (2024) prove to be an interval. The package reports the *outer* boundaries of the accepted run (so that the reported set contains the exact one) and refines an endpoint by bisection on the exact capital only when its grid cell contains a decision threshold, which is the only case in which refinement can change a decision; a nonempty set narrower than one grid cell is found by minimising the exact capital rather than reported as empty. The per-instance cost is linear in the grid size away from thresholds; a stream of 2,000 instances that never decides takes 0.3 s.

5 Simulation study

Design

The study was frozen before results were available (configuration files are dated and never edited; changes are new files) and its analysis script was written and tested on a three-replication smoke run before the study ran. All code is in `experiments/` of the package repository.¹ Losses are generated as $\text{loss}_A = u_i + \eta_{is}$ and $\text{loss}_B = u'_i + \eta'_{is} - \mu + e_{is}$ with an instance effect $u_i \sim U(0.45, 0.70)$ shared by both algorithms when paired ($u'_i = u_i$) and drawn independently otherwise, seed noise $\eta \sim U(-h_{\text{seed}}, h_{\text{seed}})$ and comparison noise $e \sim U(-h, h)$; then $E[\text{loss}_A - \text{loss}_B] = \mu$ exactly, all losses lie in $[0.05, 0.90] \subset [0, 1]$, and the standard deviation of the per-instance difference is known. The baseline grid crosses $\mu \in \{0, 0.01, 0.02, 0.05, 0.10, 0.20\}$ (null, true equivalence, exactly at the margin, moderate, large, very large effects) with $\text{sd}(D) \in \{0.03, 0.10\}$, with $\alpha = 0.05$, $\delta = 0.02$, bounds $[0, 1]$ and a budget of 2,000 instances. Two ablations use the high-noise level: unpaired instances, and five seeds per instance with seed noise 0.03. Each of the 24 cells has 2,000 replications; every method sees the same simulated stream in a given replication (seeded by cell and replication only), so method comparisons are paired. Two amendments, frozen on 2026-09-26 after an external review and run with the same methods and replication count, add 14 cells with negative effects and effects 0.005 and 0.01 away from the margin ($\mu \in \{-0.20, -0.05, -0.025, -0.015, 0.015, 0.025, 0.03\}$, both noise levels) and 12 seed-cluster cells with an instance-specific differential effect (Section 5.4). One amended cell ($\mu = -0.20$, high noise) is invalid by construction: its losses for B reach 1.07, and the package refused every replication of it rather than clip; it is reported as such and excluded, since the other 13 cells cover the negative direction.

Eight procedures are compared: the three valid boundaries and the invalid control; fixed-sample t intervals at $n = 100, 500$ and 2,000 instances with the same three-way margin rule (no adaptation); and the anytime-valid confidence sequence for the mean of `safestats'` sequential t -test (`saviTTest`,

¹<https://github.com/castlaboratory/seqbench>, outside the R package sources.

Ly et al., 2026, 2024) with the same running intersection and margin rule, its design fixed before the stream with the *known* standard deviation of the data-generating process (an oracle, favourable to safestats, which assumes Gaussian data while ours are sums of uniforms).

Metrics: false declarations (a declaration that contradicts the truth; at $\mu = \delta$ the truth is “equivalent”), coverage failure (the true mean leaves the reported running intersection at some time up to stopping), inconclusive outcomes, and instances consumed. Uncertainty: Wilson intervals for rates; percentile bootstrap over replications for medians; paired bootstrap for ratios of medians. With 2,000 replications the Monte Carlo standard error of a rate near 0.05 is 0.005; zero observed events give an upper Wilson bound of 0.0019.

Validity

Table 2 gives, for each procedure, the largest false-declaration rate and the largest coverage-failure rate over the 24 cells.

Table 2: Maximum rates over 24 cells (2,000 replications each), with the cell and the 95% Wilson interval of the maximum false-declaration rate.

Procedure	max false declaration	max coverage failure
betting	0.0045 ($\mu = 0.02$, unpaired) [0.0024, 0.0085]	0.0145
empirical Bernstein	0.0015 ($\mu = 0.02$, unpaired) [0.0005, 0.0044]	0.0025
Hoeffding	0.0000 [0, 0.0019]	0.0000
fixed $n = 100$	0.0310 ($\mu = 0.02$) [0.0243, 0.0395]	0.0610
fixed $n = 500$	0.0270 ($\mu = 0.02$) [0.0208, 0.0351]	0.0560
fixed $n = 2000$	0.0355 ($\mu = 0.02$, unpaired) [0.0282, 0.0445]	0.0640
‘safestats’ CS, oracle sd	0.0250 ($\mu = 0.02$) [0.0190, 0.0328]	0.0495
naive fixed (invalid)	0.3800 ($\mu = 0.02$) [0.3590, 0.4015]	0.6195

The three distribution-free boundaries are far inside their nominal level, with their few false declarations concentrated at the exact margin, where “equivalent” is the only correct answer and a superiority declaration is a legitimate α -event. Fixed-sample tests and the Gaussian sequence sit at their nominal level. The repeatedly inspected t -test issues a false declaration in 38% of the runs at the margin, 20% under true equivalence and 17% under the null, and its interval fails to cover the truth at some inspection in up to 62% of runs: this is the behaviour of a benchmark that peeks. Ten of its 24 cells have a Wilson lower bound above α .

Cost

Table 3: Median instances to stopping, baseline cells with $\text{sd}(D) = 0.10$, 2,000 replications. A superscript gives the percentage of inconclusive runs when above 0.5%; “budget” marks cells where more than 90% of the runs exhausted the 2,000 instances. Bootstrap 95% intervals of the medians are within 4% of the reported value in every cell.

Procedure	$\mu = 0$	$\mu = 0.01$	$\mu = 0.02$	$\mu = 0.05$	$\mu = 0.1$	$\mu = 0.2$
betting	496	1066 ^{10%}	budget	272	100	44
empirical Bernstein	920	1807 ^{36%}	budget	520	190	83
Hoeffding	budget	budget	budget	budget	budget	371
fixed $n = 100$	budget	budget	budget	100 ^{16%}	100	100
fixed $n = 500$	500 ^{1%}	500 ^{40%}	budget	500	500	500
fixed $n = 2000$	2000	2000 ^{1%}	budget	2000	2000	2000
safestats CS, oracle sd	273	760 ^{9%}	budget	83	27	17
naive fixed (invalid)	128	168	1090 ^{47%}	25	8	4

Three readings. First, adaptation pays: the betting sequence resolves the null in a median of 496 instances and the moderate effect in 272, while a fixed sample of 100 is inconclusive 96% of the time under the null and a fixed sample of 500 is inconclusive 40% of the time at $\mu = 0.01$. Second, the ordering of the distribution-free boundaries is the expected one: empirical Bernstein needs about 1.9 times the instances of betting, and Hoeffding resolves only the very large effect within the budget. Third, and this is the number a benchmarker should see before choosing: the Gaussian sequence with a known variance needs between 0.27 and 0.71 of the instances of the betting sequence in this noise

level, and between 0.05 and 0.14 in the low-noise level (Table 4), with paired bootstrap intervals of width below 0.03. The end of Section 5 explains the mechanism; Section 7 draws the consequence.

Table 4: Ratio of median instances, **safestats** CS over betting, with the paired bootstrap 95% interval, baseline cells.

sd(D)	$\mu = 0$	$\mu = 0.01$	$\mu = 0.05$	$\mu = 0.1$	$\mu = 0.2$
0.03	0.077 (0.077–0.080)	0.103 (0.099–0.109)	0.047 (0.047–0.047)	0.072 (0.072–0.072)	0.136 (0.136–0.136)
0.10	0.550 (0.536–0.562)	0.713 (0.683–0.743)	0.305 (0.297–0.312)	0.270 (0.270–0.273)	0.386 (0.378–0.386)

Pairing and seed clusters

Table 5: Betting boundary, $\text{sd}(D) = 0.10$: ratio of median instances and of mean evaluations (both algorithms, all seeds) relative to the paired single-seed design.

Ablation	$\mu = 0$	0.01	0.05	0.10	0.20
unpaired, instances	1.46	1.79	1.09	1.01	1.02
unpaired, evaluations	1.55	1.36	1.15	1.03	1.02
five seeds, instances	0.82	0.72	0.94	0.97	1.00
five seeds, evaluations	3.92	3.36	4.66	4.85	4.95

Pairing by instance costs nothing and saves up to 44% of the instances in the hard cells (and it turns a 47% inconclusive rate at $\mu = 0.01$ into 10%). Averaging five seeds per instance saves up to 28% of the *instances* but costs 3.4 to 5 times the *evaluations*: in this design the instance effect cancels exactly under pairing, so seed averaging only reduces independent seed noise, and its benefit is confined to the cells where the variance matters at all (see the truncated-bet regime below).

The companion cluster study adds to the paired difference an instance-specific term $v_i \sim U(-w, w)$ shared by all seeds of instance i , so that within-instance differences are positively correlated and row-level analysis is genuinely invalid; with $w = 0.05$ the intra-instance correlation is 0.07 and the design effect of five seeds is 1.3. Table 6 compares the correct analysis (one sequential unit per instance) with pseudoreplication (every (instance, seed) row fed as its own instance).

Table 6: Seed clusters with an instance-specific differential effect ($w = 0.05$, $\text{sd}(D) = 0.10$ per row), 2,000 replications: median instances to stopping (mean evaluations of both algorithms in parentheses; a superscript gives the percentage of inconclusive runs when above 0.5%) and the largest coverage-failure rate over the three effect sizes. For pseudoreplication the instances are the rows divided by five.

Procedure	Seeds	$\mu = 0$	$\mu = 0.01$	$\mu = 0.05$	Max. cov. failure
betting	1	518 (553)	1102 ^{14%} (1188)	275 (284)	0.30%
betting	5	413 (2108)	778 (3953)	257 (1298)	0.05%
betting, pseudoreplication	5	107 (573)	229 ^{19%} (1218)	55 (283)	1.20%
empirical Bernstein	1	937 (970)	1816 ^{39%} (1716)	530 (537)	0.00%
empirical Bernstein	5	811 (4114)	1564 ^{4%} (7857)	500 (2511)	0.00%
fixed $n = 2000$	1	2000 (2000)	2000 ^{1%} (2000)	2000 (2000)	5.40%
fixed $n = 2000$	5	2000 (10000)	2000 (10000)	2000 (10000)	5.50%

The result is a negative one that the paper reports as measured. Pseudoreplication did not produce a false declaration in 12,000 runs, and its coverage failure rose only from 0.3% (one seed) to 1.2% ($\mu = 0.01$, five seeds), four times higher but still under the nominal 5%: a design effect of 1.3 is absorbed by the slack of the betting boundary documented in Section 5.6. Its cost equals that of a single-seed analysis, because it consumes the same rows under another label. The correct five-seed analysis saves 5 to 30% of the *instances* against one seed and costs 3.3 to 4.7 times the *evaluations*, as in the first ablation. The conclusion for practice is unchanged: seeds are a cluster, not replications, and averaging them is worth its price only when instances are expensive relative to evaluations; the guarantee did not visibly break here because the correlation was mild, not because pseudoreplication is safe.

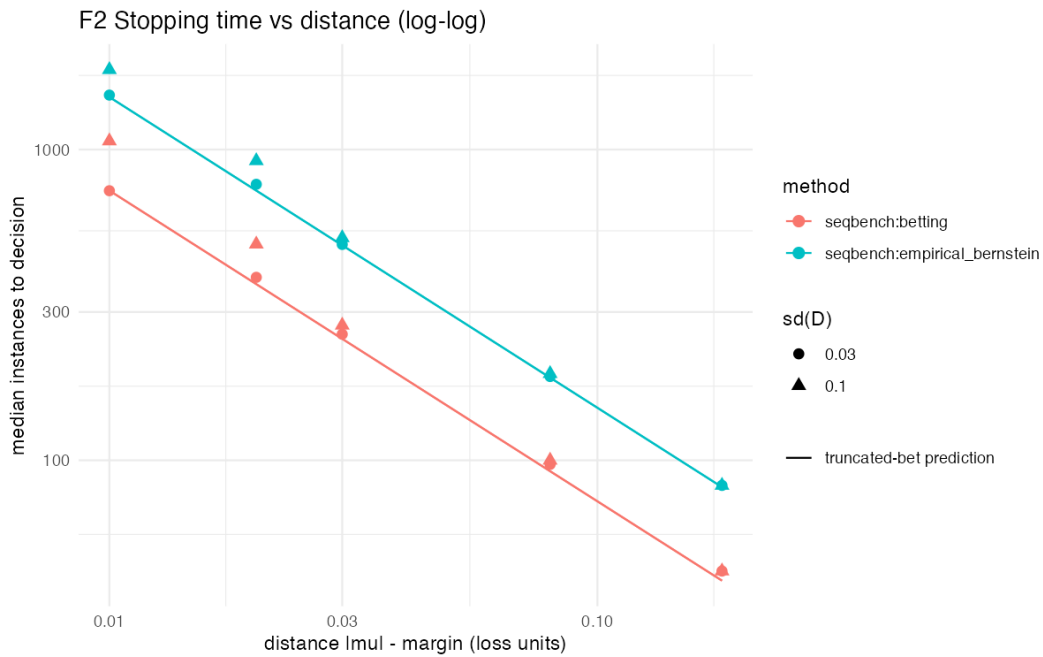


Figure 1: Median instances to decision against the distance to the nearest decision threshold (log-log), betting and empirical-Bernstein boundaries, two noise levels, 2,000 replications per point. Lines: the truncated-bet prediction $\log(2/\alpha) / (\text{cap} \times g)$.

Why the cost is what it is: the truncated-bet regime

The betting sequence bets a predictable fraction $\tilde{\lambda}_t = \sqrt{2 \log(2/\alpha) / (\hat{\sigma}_{t-1}^2 t \log(t+1))}$, truncated at c/m for the candidate mean m (with $c = 1/2$, about 1 near $m = 1/2$). While $\tilde{\lambda}_t$ exceeds the cap, which happens while $t \log(t+1) < 2 \log(2/\alpha) / (\hat{\sigma}^2 \cdot \text{cap}^2)$, the bet is constant, the log-capital at a candidate at distance g from the truth grows at about $\text{cap} \cdot g$ per instance, and the stopping time is approximately

$$\tau \approx \frac{\log(2/\alpha)}{\text{cap} \cdot g} = \frac{\log(2/\alpha) (b-a)}{\text{cap} \cdot ||\mu| - \delta|},$$

linear in the inverse distance to the nearest threshold and *independent of the variance*. Table 7 compares this prediction with the observed medians.

Table 7: Betting boundary, median instances observed and predicted by the truncated-bet approximation; $\text{sd}(D) = 0.03$ (crossover at $t \approx 4,060$) and 0.10 (crossover at $t \approx 480$).

μ	distance	observed, sd 0.03	predicted	observed, sd 0.10
0.00	0.02	388	369	496
0.01	0.01	737	738	1066
0.05	0.03	254	246	272
0.10	0.08	97	92	100
0.20	0.18	44	41	44

At the low noise level the prediction is within 7% everywhere; at the high level it holds where the decision comes before the crossover and degrades to 1.34–1.44 of the prediction where it comes after, when the variance term of the bet becomes active. Figure 1 shows the observed medians against the prediction. A log-log regression of the median stopping time on distance and standard deviation over the resolved cells gives slopes -1.04 (s.e. 0.02) and $+0.10$ (0.03).

Two practical consequences: in low-noise benchmarks with a tight margin, halving the distance to the threshold doubles the cost rather than quadrupling it; and the boundary cannot exploit small variance beyond its bet cap, which is exactly why the Gaussian sequence, which bets on g/σ^2 , is so much faster when σ is small. Raising the cap helps and costs nothing measurable: in a sweep on the twelve baseline cells (2,000 replications each, same streams), $c = 0.9$ needed 222 instead of 388 instances at the null and 143 instead of 254 at $\mu = 0.05$ for $\text{sd}(D) = 0.03$ (385 and 165 instead of 496 and 272 at $\text{sd}(D) = 0.10$), with the largest false-declaration rate rising from 0.20% to 0.70% and the

largest coverage failure from 0.55% to 1.40%, both far below α ; $c = 0.99$ gains little more. The rest of the gap to the Gaussian sequence is not recoverable on a bounded support, as Section 7 explains.

Time-uniform coverage without stopping

Coverage in Table 2 is measured up to the stopping time. A separate audit ran each boundary over the full stream of 2,000 instances on the same 24 cells (same seeds, hence the same streams; for the betting boundary the exact capital at the true mean is evaluated, which needs no grid) and recorded whether the true mean ever left the running confidence sequence. Over 2,000 replications per cell the largest failure rate was 1.55% for betting, 0.25% for empirical Bernstein and 0 for Hoeffding, against the nominal 5%: the time-uniform guarantee holds with room to spare, which is the other face of the cost measured above.

Negative effects and effects near the margin

The first amendment fills the two gaps of the baseline grid: effects in the direction of A, and effects 0.005 and 0.01 away from a decision threshold ($|\mu| \in \{0.015, 0.025, 0.03\}$ with $\delta = 0.02$). Over its 13 valid cells no valid boundary made a false declaration (upper Wilson bound 0.19% per cell); the *safestats* sequence reached 0.95% and the repeatedly inspected *t*-test 22.8% ($\mu = -0.015$, high noise). Coverage failure up to stopping was at most 0.25% for betting and 0 for the other two. Table 8 gives the cost.

Table 8: Median instances to stopping in the amended cells (μ in the column heads; $\delta = 0.02$), 2,000 replications, same conventions as Table 3; “—” marks the cell invalid by construction.

sd(<i>D</i>)	Procedure	−0.2	−0.05	−0.025	−0.015	0.015	0.025	0.03
0.03	betting	44	255	1552 ^{5%}	1496 ^{4%}	1497 ^{3%}	1562 ^{6%}	769
	empirical Bernstein	83	495	budget	budget	budget	budget	1503
	Hoeffding	372	budget	budget	budget	budget	budget	budget
	fixed $n = 500$	500	500	500 ^{4%}	500 ^{3%}	500 ^{4%}	500 ^{4%}	500
	<i>safestats</i> CS, oracle sd	6	12	419	399	404	431	81
0.10	betting	—	269	2000 ^{80%}	2000 ^{81%}	2000 ^{79%}	2000 ^{80%}	1042 ^{10%}
	empirical Bernstein	—	521	budget	budget	budget	budget	1755 ^{33%}
	Hoeffding	—	budget	budget	budget	budget	budget	budget
	fixed $n = 500$	500	500	500 ^{80%}	500 ^{79%}	500 ^{80%}	500 ^{79%}	500 ^{38%}
	<i>safestats</i> CS, oracle sd	17	85	2000 ^{75%}	2000 ^{76%}	2000 ^{75%}	2000 ^{75%}	714 ^{8%}

Two readings. The procedures are symmetric in the sign of μ , as they should be: the betting boundary needs 255 and 269 instances at $\mu = -0.05$ against 254 and 272 at $+0.05$. And 0.005 from a threshold is beyond every procedure at the high noise level, the Gaussian oracle included: *safestats* is inconclusive in 75% of the runs and betting in 80%, against 98% for a fixed sample of 100 whose interval is simply too wide to declare. At the low noise level betting resolves these cells in a median of about 1,500 instances with 3 to 6% inconclusive runs, and the Gaussian sequence in about 400, the 0.27 ratio of Table 4 again. The planning function of Section 3.3 returns exactly this horizon before the first instance is run, which is where a benchmarker should learn that a distance of 0.005 is not a question the budget can answer.

6 Applications

All three applications use the workflow of Section 4 with the same reference procedures. The truth is unknown, so they demonstrate usability and cost, not error rates.

Two estimators of location

Instances are samples of size 20, 40 or 80 from a contaminated normal with scale $\sim U(1, 3)$ and contamination fraction $\sim U(0, 0.25)$, drawn per instance; *A* is the sample mean, *B* the 20% trimmed mean; the loss is the absolute error divided by the scale, capped at 1; $\delta = 0.02$; 500 replications of streams of up to 3,000 instances. A Monte Carlo reference over 100,000 instances puts μ at 0.086. All procedures declared the trimmed mean relevantly better in every replication; the median number of

instances was 81 for the safestats sequence (pilot-estimated sd), 128 for betting, 246 for empirical Bernstein, and 3,000 for the fixed-sample test, which only decides at the end of its budget.

Two knapsack heuristics

Instances are random 0/1 knapsack problems with 50 to 200 items; A is the greedy heuristic by value-to-weight ratio, B a GRASP-like randomised greedy with local search and 20 restarts; the loss is the relative gap to the Dantzig linear-programming bound, truncated at 5% and rescaled to $[0, 1]$, so that the margin $\delta = 0.02$ is 0.1 percentage points of gap (the truncation is the operational choice of Section 2, made because raw gaps of 0.1–1% would waste the $[0, 1]$ scale); the cost is the number of objective evaluations; 100 replications of up to 600 instances. The betting sequence declared the GRASP-like heuristic relevantly better in every replication after a median of 397 instances (76 million evaluations); the fixed-sample test declared the same at 600; the empirical Bernstein sequence was inconclusive at 600 in 97% of the replications, its interval $[0.015, 0.066]$ still straddling the margin.

Louvain versus Leiden

Instances are stochastic block models with 3 to 6 blocks of unequal sizes, 80 to 300 nodes and per-instance mixing; A is `igraph` (Csárdi et al., 2026) `cluster_louvain()`, B `cluster_leiden()` run to convergence on modularity, both with the instance's seed; the loss is $1 - Q$ with bounds $[0, 1.5]$; $\delta = 0.01$ (one modularity point); 100 replications of up to 6,000 instances. A Monte Carlo reference over 5,000 instances puts the mean difference at 0.0055 in favour of Leiden, inside the margin, and the per-instance differences are far from Gaussian: 40% exact ties, Louvain ahead in 3.5% of the instances, skewness 1.8. All four procedures declared practical equivalence in every replication. The betting sequence did so after a median of 2,533 instances, empirical Bernstein after 5,090, the fixed-sample test at its limit of 6,000, and the safestats sequence after a median of 26. That last number is not a success. On this distribution the Gaussian sequence, fed a pilot standard deviation, stops after 4 or 5 instances whenever the first differences are all ties, with an interval that has collapsed to a point; its running interval excluded the reference mean in 15% of the replications (against 0 for the two bounded boundaries; the fixed-sample “failures” are not interpretable, since at $n = 6,000$ its interval is as narrow as the reference's own Monte Carlo error). The declaration happened to be right because the truth is equivalence, but the guarantee was gone. This is the case for distribution-free validity in one picture: algorithm outputs tie, cluster and skew, and a sequence that assumes a model pays for its speed in coverage exactly where the assumption fails.

7 Discussion

What the package buys. A benchmark that peeks, adapts and stops early keeps its error guarantee, at a stated level, with an auditable record of what was assumed and consumed, and with “we could not tell within the budget” as a legitimate outcome. The simulation study puts numbers on the failure this prevents (38% false declarations for the peeking t -test) and on what adaptation saves relative to fixed samples.

What it costs. Distribution-free validity for bounded losses is expensive when the losses have small variance relative to their range: the Gaussian sequence with a known variance needed 5 to 20 times fewer instances at the low noise level. The mechanism is the bet cap, and the consequence is a floor on the cost of bounded-only validity that does not vanish as the variance shrinks. That floor is a theorem, not an artefact of the boundaries used here: a standard change-of-measure argument against a small contamination at an endpoint of the loss range shows that any procedure valid for every bounded distribution needs, in expectation, at least of the order of $\log(1/\alpha)/g$ instances, whatever the variance; the betting sequence with its cap near 1 stays within a factor of about 1.5 of it in every cell of the simulation study (a companion paper, in preparation, gives the statement and proof). Two levers are available now: declare bounds as tight as the loss allows (truncating a gap at a sensible cutoff, as in the knapsack application, is such a declaration) and raise the bet cap towards 1. A declared variance bound did not help much with the construction implemented in the package: the `bernstein_declared` boundary, a predictable-plug-in Bennett sequence that takes a declared upper bound on the standard deviation, needed 220 instances at the low-noise null where betting with $c = 0.9$ needed 222 (and 968 against 385 at the high-noise null), because Bennett's bound keeps a range term of order $\log(2/\alpha)/t$; with the bound declared at half its true value it was faster and still covered in more than 98% of the replications, which says how loose the bound is rather than how safe the violation would be in general. This is a property of the Bennett construction, not a proved limit: the floor above does not extend to procedures that assume a variance bound, and whether one can do markedly better on bounded data

is open. The Gaussian sequence escapes the floor by assuming a symmetric, light-tailed model, and what breaks it is skewness: in a further simulation with bounded paired differences its coverage failed in 13% of the replications when the differences had skewness 1.9 and no ties, while 70% ties without skewness left it intact; the betting and empirical-Bernstein sequences did not fail in any of those cells. The Louvain–Leiden application shows the same bill on real algorithm outputs.

What it does not claim. The error control is a margin rule on an existing confidence sequence, obtainable from `safestats` or from the constructions of Choe and Ramdas (2024) by a user who knows what to do; sequential equivalence with e-processes is developed by Koobs and Koning (2026). The package’s contribution is the protocol, its refusals, its contract and the honest accounting.

Limitations. Two algorithms only (multiplicity across $k > 2$ needs a correction or the model confidence sets of Arnold et al. (2026)); instances as the sequential unit (adding seeds adaptively within an instance is refused rather than modelled); bounded losses; and a population target, which is the right one for “problems like these” and the wrong one for a fixed benchmark suite.

8 Summary

seqbench turns an algorithm comparison into a pre-registered sequential experiment with an anytime-valid guarantee, a margin of practical equivalence and an honest account of cost. Version 0.1.0 is available from CRAN;² the source is on GitHub³ and the documentation on the package site.⁴

Bibliography

- A. Alasgarli. *seqcomp: Sequential Comparison of Probabilistic Forecasts*, 2026. URL <https://CRAN.R-project.org/package=seqcomp>. R package version 0.1.0. [p1]
- P. Armitage, C. K. McPherson, and B. C. Rowe. Repeated significance tests on accumulating data. *Journal of the Royal Statistical Society: Series A*, 132(2):235–244, 1969. doi: 10.2307/2343787. [p1]
- S. Arnold, G. Gavrilopoulos, B. Schulz, and J. Ziegel. Sequential model confidence sets. *Journal of the Royal Statistical Society: Series B*, 2026. doi: 10.1093/jrsssb/qkag066. [p10]
- T. Bartz-Beielstein, C. Doerr, D. van den Berg, J. Bossek, S. Chandrasekaran, T. Eftimov, A. Fischbach, P. Kerschke, W. La Cava, M. Lopez-Ibanez, K. M. Malan, J. H. Moore, B. Naujoks, P. Orzechowski, V. Volz, M. Wagner, and T. Weise. Benchmarking in optimization: Best practice and open issues. arXiv:2007.03488, 2020. [p1]
- A. Benavoli, G. Corani, J. Demšar, and M. Zaffalon. Time for a change: A tutorial for comparing multiple classifiers through Bayesian analysis. *Journal of Machine Learning Research*, 18(77):1–36, 2017. [p2]
- M. Birattari, T. Stützle, L. Paquete, and K. Varrentrapp. A racing algorithm for configuring metaheuristics. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2002)*, pages 11–18. Morgan Kaufmann, 2002. [p1]
- Y. J. Choe and A. Ramdas. Comparing sequential forecasters. *Operations Research*, 72(4):1368–1387, 2024. doi: 10.1287/opre.2021.0792. [p1, 10]
- G. Csárdi, T. Nepusz, V. Traag, S. Horvát, F. Zanini, D. Noom, and K. Müller. *igraph: Network Analysis and Visualization in R*, 2026. URL <https://CRAN.R-project.org/package=igraph>. R package version 2.3.3. [p9]
- T. Eimer, L. Schäpermeier, A. Biedenkapp, et al. Best practices for empirical meta-algorithmic research: Guidelines from the COSEAL research network. arXiv:2512.16491, 2025. [p1]
- S. R. Howard, A. Ramdas, J. McAuliffe, and J. Sekhon. Time-uniform, nonparametric, nonasymptotic confidence sequences. *The Annals of Statistics*, 49(2):1055–1080, 2021. doi: 10.1214/20-AOS1991. [p1]
- F. Hutter, H. H. Hoos, K. Leyton-Brown, and T. Stützle. ParamILS: An automatic algorithm configuration framework. *Journal of Artificial Intelligence Research*, 36:267–306, 2009. [p2]

²<https://CRAN.R-project.org/package=seqbench>

³<https://github.com/castlaboratory/seqbench>

⁴<https://castlaboratory.github.io/seqbench/>

- S. Koobs and N. W. Koning. Equivalence testing with data-dependent and post-hoc equivalence margins. *arXiv:2603.16213*, 2026. [p10]
- D. Lakens. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4):355–362, 2017. doi: 10.1177/1948550617697177. [p2]
- M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, and T. Stützle. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016. doi: 10.1016/j.orp.2016.09.002. [p1]
- A. Ly, U. Boehm, P. Grünwald, A. Ramdas, and D. van Ravenzwaaij. Safe anytime-valid inference: Practical maximally flexible sampling designs for experiments based on e-values. *PsyArXiv preprint*, 2024. [p1, 5]
- A. Ly, R. Turner, J. ter Schure, M. F. Pérez-Ortiz, and P. Grünwald. *safestats: Safe Anytime-Valid Inference*, 2026. URL <https://CRAN.R-project.org/package=safestats>. R package version 0.8.8. [p1, 5]
- A. Ramdas, P. Grünwald, V. Vovk, and G. Shafer. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science*, 38(4):576–601, 2023. doi: 10.1214/23-STS894. [p1]
- D. J. Schuirmann. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6):657–680, 1987. doi: 10.1007/BF01068419. [p2]
- I. Waudby-Smith and A. Ramdas. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society: Series B*, 86(1):1–27, 2024. doi: 10.1093/jrsssb/qkad009. [p1, 2, 4]

André Leite
CAST Lab, Universidade Federal de Pernambuco
Departamento de Estatística
Recife, Brazil
<https://castlab.org>
leite@castlab.org

Raydonal Ospina
Universidade Federal da Bahia
Departamento de Estatística
Salvador, Brazil
<https://castlab.org>
raydonal@castlab.org

Cristiano Ferraz
CAST Lab, Universidade Federal de Pernambuco
Departamento de Estatística
Recife, Brazil
<https://castlab.org>
cferraz@castlab.org